

SHL Assessment Recommendation Assistant — Approach Document

Author: Sanket Shakya

1. Solution Overview

The objective of this project was to build a conversational AI assistant that recommends the most relevant SHL assessments based on hiring requirements or job descriptions. The system uses a Retrieval-Augmented Generation (RAG) architecture, where semantic search retrieves relevant assessments from the SHL catalog before an LLM generates grounded explanations, reducing hallucinations and ensuring recommendations are limited to official SHL assessments.

The solution consists of a FastAPI backend, Qdrant Cloud vector database, Gemini Embedding API for semantic retrieval, Gemini 2.5 Flash for reasoning, and a Streamlit interface for demonstration.

2. Design Choices

- **FastAPI** — Lightweight REST API framework for /health and /chat.
- **Gemini Embedding API (gemini-embedding-001, 768-dim)** — Generates semantic embeddings. Chosen over local sentence-transformers to eliminate ~1GB of torch/transformers dependencies and stay within Render free tier limits.
- **Qdrant Cloud** — Stores 768-dim assessment embeddings; performs cosine similarity search.
- **Gemini 2.5 Flash** — Produces natural language explanations grounded in retrieved assessments.
- **Streamlit** — Interactive interface for demonstrating the assistant.

A RAG architecture was chosen instead of direct LLM prompting because recommendations must always originate from the official SHL catalog.

3. Retrieval Setup

The SHL catalog (377 assessments) was converted into structured JSON with metadata: name, description, categories, job levels, duration, remote/adaptive support, and official URL. Each assessment is embedded via the Gemini Embedding API and indexed in Qdrant Cloud.

Flow: query embedded → Qdrant retrieves top-10 by cosine similarity → retrieval scores attached → passed to Gemini for explanation. Final recommendations are sorted by retrieval score descending, so order always reflects semantic relevance regardless of LLM re-ranking.

4. Prompt Design

Gemini receives conversation history, latest user request, retrieved assessments, and retrieval scores. It is instructed to:

- Use only retrieved assessments.
- Never invent assessment names or URLs.
- Explain why each retrieved assessment matches the requirement.
- Prefer assessments with higher retrieval scores when appropriate.

Gemini returns structured JSON with only the summary and explanations; all metadata (URL, duration, remote, adaptive) is supplied by the backend from the catalog.

5. Conversation Handling

- **Clarification** — Requests role, skills, seniority, or assessment type for vague queries.
- **Recommendation** — Suggests 1–10 assessments with reasons and retrieval scores.
- **Refinement** — Uses full conversation history to update recommendations as constraints change.
- **Comparison** — Compares two assessments using catalog metadata only.
- **Prompt Injection Detection** — Detects and refuses attempts to redirect the assistant outside the SHL catalog.
- **Out-of-Scope Handling** — Politely declines unrelated requests.

The API is stateless; the client sends full conversation history each request, enabling multi-turn refinement without server-side sessions.

6. Evaluation

Evaluated using representative hiring scenarios: Java Backend Developer, Python Developer, Data Scientist, Sales Executive, and Graduate Hiring. Additional testing covered clarification, refinement, comparison, prompt injection attempts, and out-of-scope queries.

Focus areas: recommendation relevance, catalog grounding, conversational correctness, consistent ranking by retrieval score, and schema consistency across response types.

7. Challenges & Future Improvements

Primary challenge: preventing LLM hallucinations, addressed by restricting Gemini strictly to retrieved assessments with backend-supplied metadata. Secondary challenge: deployment size — the original torch/sentence-transformers stack (~1GB) exceeded Render free tier limits and caused cold-start timeouts, resolved by switching to the Gemini Embedding API.

Future improvements: hybrid keyword + semantic retrieval, cross-encoder reranking, fuzzy name resolution for comparisons, automated evaluation metrics, streaming responses.

8. AI Tools Used

- **ChatGPT (OpenAI)** — Architecture discussions, debugging, prompt refinement, implementation guidance.
- **Gemini 2.5 Flash** — Recommendation generation and assessment comparison.
- **Gemini Embedding API (gemini-embedding-001)** — Semantic embedding for catalog indexing and query embedding.
- **Qdrant Cloud** — Vector similarity search and retrieval.

Architecture Diagram

